



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Punkt kontrolny 1

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 26 marca 2026

1 Cel projektu

Zadanie polega na zaprojektowaniu dwóch formalnych języków dla klasy *DS4* systemów dynamicznych:

- *języka akcji* — służącego do opisu dziedziny (fluenty, akcje, ich skutki, ograniczenia),
- *języka kwerend* — pozwalającego zadawać pytania o wykonywalność procesów i osiągalność celów.

Klasa *DS4* łączy cechy kilku języków akcji znanych z wykładów. Mechanizmy inercji, niedeterminizmu, ramifikacji i niewykonalności pochodzą z języka *AR* (wykład 2). Obsługę akcji złożonych projektujemy w oparciu o język *AC* (wykład 5) — adaptujemy mechanizm detekcji konfliktów i dekompozycji do warunków *DS4*, w których opisy dziedzin zawierają wyłącznie akcje proste.

2 Język akcji

Zdanie	Co oznacza
$A \text{ causes } \alpha \text{ if } \pi$	Akcja A powoduje efekt α , o ile zachodzi warunek π .
$A \text{ releases } f \text{ if } \pi$	Akcja A „zwalnia” fluent f — może, ale nie musi się zmienić (niedeterminizm).
$\text{impossible } A \text{ if } \pi$	Akcja A jest niewykonalna, gdy zachodzi π .
$\text{always } \alpha$	Formuła α musi być prawdziwa w każdym stanie (ograniczenie).
$\text{noninertial } f$	Fluent f nie podlega inercji — może się zmieniać jako skutek uboczny.
$\text{initially } \alpha$	Na początku zachodzi α .
$\alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji na pewno zachodzi α .
$\text{observable } \alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji może zachodzić α .

Wszystkie powyższe zdania dotyczą wyłącznie akcji prostych $A \in \mathcal{A}$. Akcje złożone pojawiają się dopiero w języku kwerend (w procesach).

3 Semantyka

Stany

Stan to przypisanie wartości prawda/fałsz każdemu fluentowi. Nie każdy stan jest dopuszczalny — odrzucamy te, które łamią ograniczenia **always**. Na początku bierzemy stany dopuszczalne spełniające zdania **initially** i obserwacje — to zbiór stanów początkowych Σ_0 (może ich być wiele, jeśli opis jest częściowy).

Wykonanie akcji prostej

Żeby obliczyć, co się dzieje po wykonaniu akcji A w stanie σ :

1. Sprawdzamy, czy A nie jest niewykonalna (czy nie ma aktywnego **impossible**).
2. Zbieramy efekty — patrzymy, które zdania **causes** mają spełniony warunek wstępny w σ , i łączymy ich efekty.
3. Szukamy stanów dopuszczalnych, w których te efekty zachodzą.

4. Spośród nich wybieramy te, które zmieniły się *jak najmniej* w porównaniu do σ (inercja).
 Fluenty **noninertial** nie liczą się przy tym porównaniu — mogą się zmieniać „za darmo”.

Wynikiem może być kilka stanów (niedeterminizm z **releases**), jeden stan, albo żaden (niewykonalność).

Konflikty między akcjami

Pojęcie konfliktu adaptujemy z języka *AC* (wykład 5). Przez $\text{DNF}(\alpha)$ oznaczamy dysjunkcyjną postać normalną formuły α (alternatywa koniunkcji literalów).

Dwie akcje proste $A, B \in \mathcal{A}$ są w konflikcie w stanie σ , jeśli zachodzi co najmniej jeden z warunków:

Przypadek 1 (causes vs. causes). Istnieją zdania (A **causes** α **if** π) i (B **causes** β **if** ψ) takie, że $\sigma \models \pi \wedge \psi$ oraz pewien składnik $\text{DNF}(\alpha)$ zawiera literal komplementarny do literalu w pewnym składniku $\text{DNF}(\beta)$.

Przypadek 2 (causes vs. releases). Istnieją zdania (A **causes** α **if** π) i (B **releases** f **if** ψ) (lub odwrotnie) takie, że $\sigma \models \pi \wedge \psi$ oraz f lub $\neg f$ występuje w pewnym składniku $\text{DNF}(\alpha)$.

Intuicyjnie: dwie akcje kolidują, gdy mogą wymusić sprzeczne wartości tego samego fluenta — albo bezpośrednio (Przypadek 1), albo jedna wymusza wartość, a druga ją „zwalnia” (Przypadek 2).

Akcje złożone — dekompozycja

Akcja złożona $\mathbb{A} = \{A_1, \dots, A_k\} \subseteq \mathcal{A}$ to zbiór akcji prostych wykonywanych równocześnie. Jeśli pewne składowe kolidują ze sobą, nie wykonujemy wszystkich naraz — zamiast tego szukamy *maksymalnych* podzbiorów, które można bezpiecznie uruchomić razem. Mechanizm ten adaptujemy z *AC* (wykład 5).

Definicja. Podzbiór $S \subseteq \mathbb{A}$ jest *dekompozycją* akcji złożonej $\mathbb{A}$ w stanie σ , gdy:

- (D1) Każda akcja $A_i \in S$ jest indywidualnie wykonalna w σ (brak aktywnego **impossible**).
- (D2) Żadne dwie różne akcje $A_i, A_j \in S$ nie są w konflikcie w σ .
- (D3) S jest *maksymalny* — nie da się dołożyć żadnej akcji z $\mathbb{A} \setminus S$ bez naruszenia (D1) lub (D2).

Dekompozycji może być kilka — oznaczamy je $\Delta_1(\mathbb{A}, \sigma), \dots, \Delta_n(\mathbb{A}, \sigma)$. Istnienie wielu dekompozycji jest nowym, niezależnym źródłem niedeterminizmu (obok **releases**).

Jeśli żadna akcja ze składowych nie jest indywidualnie wykonalna, dekompozycja nie istnieje i $\text{Res}(\mathbb{A}, \sigma) = \emptyset$. Jeśli żadna para akcji nie koliduje, jedyną dekompozycją to $S = \mathbb{A}$ i obliczamy wynik tak jak dla akcji prostej — łącząc efekty wszystkich składowych.

Ponieważ w *DS4* dziedzina opisuje tylko akcje proste, nie pojawia się mechanizm „ograniczonego dziedziczenia” (restricted inheritance) z pełnego *AC* — efekty akcji prostych są zawsze dziedziczone w całości.

Stany wynikowe akcji złożonej

Dla każdej dekompozycji Δ_i wynik obliczamy osobno — zbieramy aktywne efekty akcji z Δ_i , szukamy pasujących stanów dopuszczalnych i stosujemy minimalizację (inercja), tak jak dla akcji prostej. Wynik końcowy to suma wyników ze wszystkich dekompozycji:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{i=1}^n \text{Res}(\Delta_i, \sigma).$$

Procesy

Proces $P = (\mathbb{A}_1, \dots, \mathbb{A}_n)$ to ciąg akcji złożonych wykonywanych jedna po drugiej. Zbiór stanów osiągalnych po wykonaniu procesu P ze stanu σ definiujemy rekurencyjnie:

$$\Psi(\varepsilon, \sigma) = \{\sigma\}, \quad \Psi((\mathbb{A}_1, \dots, \mathbb{A}_k), \sigma) = \bigcup_{\sigma' \in \Psi((\mathbb{A}_1, \dots, \mathbb{A}_{k-1}), \sigma)} \text{Res}(\mathbb{A}_k, \sigma').$$

Na każdym kroku rozpatrujemy wszystkie stany z poprzedniego kroku i zbieramy wyniki ze wszystkich ścieżek.

4 Język kwerend

Język kwerend pozwala zadawać dwa rodzaje pytań o procesy:

- **Czy proces jest wykonywalny?**
 - **necessary executable** P — „Czy P zawsze da się wykonać, niezależnie od stanu początkowego i niedeterminizmu?”
 - **possibly executable** P — „Czy istnieje choć jedna ścieżka, na której cały P da się wykonać?”
- **Czy po procesie osiągamy cel?**
 - **necessary γ after** P — „Czy γ zachodzi *zawsze* po wykonaniu P ?”
 - **possibly γ after** P — „Czy *istnieje* ścieżka wykonania P , po której zachodzi γ ?”

necessary oznacza „dla każdego możliwego przebiegu”, a **possibly** — „istnieje taki przebieg”.

5 Przykłady

5.1 YSP z rosyjską ruletką

Scenariusz oparty na rosyjskim indyku (wykład 2), ilustrujący niedeterminizm (**releases**), niewykonalność (**impossible**) i częściowy opis stanu początkowego.

Sygatura: $\mathcal{F} = \{\text{loaded}, \text{alive}\}$, $\mathcal{A} = \{\text{Load}, \text{Shoot}, \text{Spin}\}$.

Dziedzina D :

initially <i>alive</i>	(stan broni nieznany — opis częściowy)
<i>Load</i> causes <i>loaded</i>	
impossible <i>Load</i> if <i>loaded</i>	(nie można ładować naładowanej)
<i>Shoot</i> causes $\neg$ <i>loaded</i>	
<i>Shoot</i> causes $\neg$ <i>alive</i> if <i>loaded</i>	
<i>Spin</i> releases <i>loaded</i>	(kręcenie bębenkiem — niedeterminizm)

Stany dopuszczalne (brak **always**, wszystkie 4 stany dopuszczalne):

$$\sigma_0 = \{\text{loaded}, \text{alive}\}, \quad \sigma_1 = \{\text{loaded}, \neg \text{alive}\}, \quad \sigma_2 = \{\neg \text{loaded}, \text{alive}\}, \quad \sigma_3 = \{\neg \text{loaded}, \neg \text{alive}\}$$

$\Sigma_0 = \{\sigma_0, \sigma_2\}$ — dwa możliwe stany początkowe, bo *loaded* nie jest opisane w zdaniu **initially**.

Proces $P = (\{\text{Load}\}, \{\text{Spin}\}, \{\text{Shoot}\})$.

Z σ_0 : *Load* jest niewykonalne (bo $\sigma_0 \models \text{loaded}$, a **impossible** *Load* **if** *loaded*). Proces nie może startować z σ_0 .

Z σ_2 :

1. *Load* w σ_2 : efekt $loaded \Rightarrow \sigma_0 = \{loaded, alive\}$.
2. *Spin* w σ_0 : fluent $loaded$ jest zwolniony (**releases**). Oba stany σ_0 i σ_2 zmieniają się minimalnie, więc $Res(Spin, \sigma_0) = \{\sigma_0, \sigma_2\}$ (niedeterminizm).
3. *Shoot* po każdej gałęzi:
 - ze σ_0 (broń naładowana): efekty $\neg loaded \wedge \neg alive \Rightarrow \sigma_3$.
 - ze σ_2 (broń pusta): efekt $\neg loaded$ już spełniony, warunek **if** $loaded$ fałszywy $\Rightarrow$ efekt pusty, σ_2 bez zmian.

Stany końcowe: $\{\sigma_2, \sigma_3\}$.

Kwerendy:

- **necessary executable** P — z $\sigma_0 \in \Sigma_0$ pierwszy krok jest niewykonalny. **NIE**.
- **possibly executable** P — z σ_2 proces przechodzi wszystkie kroki. **TAK**.
- **necessary $\neg alive$ after** P — gałąź przez σ_2 (po *Spin*) kończy się w σ_2 z *alive*. **NIE**.
- **possibly $\neg alive$ after** P — gałąź przez σ_0 (po *Spin*) kończy się w σ_3 . **TAK**.

5.2 Producent i konsumenci

Scenariusz oparty na przykładzie 5.8 z wykładu 5. W odróżnieniu od oryginału nie blokujemy sytuacji, w której obaj konsumenci pobierają produkt jednocześnie — w *DS4* nie można wyrazić **impossible** dla akcji złożonej. Daje to dodatkową dekompozycję i dodatkowy niedeterminizm. Producent napenia bufor, dwaj konsumenci A i B pobierają z niego produkt i zużywają go.

Sygnatura: $\mathcal{F} = \{empty, hasA, hasB\}$, $\mathcal{A} = \{Put, GetA, GetB, ConsumeA, ConsumeB\}$.

Dziedzina D :

initially $\neg empty \wedge hasA \wedge hasB$
Put **causes** $\neg empty$
GetA **causes** $empty \wedge hasA$ **if** $\neg empty$
GetB **causes** $empty \wedge hasB$ **if** $\neg empty$
ConsumeA **causes** $\neg hasA$ **if** $hasA$
ConsumeB **causes** $\neg hasB$ **if** $hasB$

Brak **always** — wszystkie $2^3 = 8$ stanów jest dopuszczalnych. Stan początkowy: $\Sigma_0 = \{\sigma\}$ gdzie $\sigma = \{\neg empty, hasA, hasB\}$ (bufor pełny, obaj konsumenci mają produkt).

Akcja złożona $\mathbb{A} = \{Put, GetA, GetB, ConsumeA, ConsumeB\}$ w σ .

Konflikty w σ (oba warunki wstępne spełnione):

- *Put* vs. *GetA*: *Put* daje $\neg empty$, *GetA* daje $empty \wedge hasA$ — literały $\neg empty$ i $empty$ są komplementarne. **Konflikt** (Przypadek 1).
- *Put* vs. *GetB*: analogicznie. **Konflikt**.
- *GetA* vs. *ConsumeA*: *GetA* daje $empty \wedge hasA$, *ConsumeA* daje $\neg hasA$ — $hasA$ i $\neg hasA$ komplementarne. **Konflikt**.
- *GetB* vs. *ConsumeB*: analogicznie. **Konflikt**.

- *GetA* vs. *GetB*: oba dają *empty* (ten sam znak) i różne fluenty *hasA*, *hasB* — **brak konfliktu**. W oryginalnym przykładzie 5.8 z wykładu *GetA* i *GetB* są wykluczone zdaniem **impossible** $\{GetA, GetB\}$. W *DS4 impossible* dotyczy tylko akcji prostych, więc tej pary nie możemy tak zablokować.
- *ConsumeA* vs. *ConsumeB*, *Put* vs. *ConsumeA/ConsumeB*, *GetA* vs. *ConsumeB*, *GetB* vs. *ConsumeA*: brak konfliktu.

Dekompozycje — maksymalne podzbiory bez konfliktu:

- $\Delta_1 = \{Put, ConsumeA, ConsumeB\}$ — nie da się dołożyć *GetA* ani *GetB* (oba kolidują z *Put*).
- $\Delta_2 = \{GetA, GetB\}$ — nie da się dołożyć *Put*, *ConsumeA* ani *ConsumeB* (konflikty). *Ta dekompozycja nie istnieje w oryginalnym przykładzie 5.8.*
- $\Delta_3 = \{GetA, ConsumeB\}$ — nie da się dołożyć *Put*, *GetB* ani *ConsumeA* (konflikty).
- $\Delta_4 = \{GetB, ConsumeA\}$ — analogicznie.

Wyniki z $\sigma = \{\neg empty, hasA, hasB\}$:

- $\Delta_1: \neg empty \wedge \neg hasA \wedge \neg hasB \Rightarrow \sigma_1$.
- $\Delta_2: empty \wedge hasA \wedge hasB \Rightarrow \sigma_2$.
- $\Delta_3: empty \wedge hasA \wedge \neg hasB \Rightarrow \sigma_3$.
- $\Delta_4: empty \wedge \neg hasA \wedge hasB \Rightarrow \sigma_4$.

$Res(\mathbb{A}, \sigma) = \{\sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Wykład 5 daje tylko $\{\sigma_1, \sigma_3, \sigma_4\}$ — brak σ_2 , bo tam *GetA* i *GetB* są razem niemożliwe.

Proces $P = (\mathbb{A}, \{ConsumeA, ConsumeB\})$.

Krok 2 — $\{ConsumeA, ConsumeB\}$ w każdym stanie wynikowym (brak konfliktu, bo $\neg hasA$ i $\neg hasB$ dotyczą różnych fluentów):

- $\sigma_1 = \{\neg e, \neg A, \neg B\}$: oba warunki (*hasA*, *hasB*) niespełnione — efekt pusty, stan bez zmian.
- $\sigma_2 = \{e, A, B\}$: obaj konsumenci konsumują $\Rightarrow \sigma_0 = \{e, \neg A, \neg B\}$.
- $\sigma_3 = \{e, A, \neg B\}$: tylko A konsumuje $\Rightarrow \sigma_0$.
- $\sigma_4 = \{e, \neg A, B\}$: tylko B konsumuje $\Rightarrow \sigma_0$.

(Używamy skrótu $e = empty$, $A = hasA$, $B = hasB$.)

Po procesie P : $Res = \{\sigma_0, \sigma_1\}$.

Kwerendy:

- **necessary executable** P — oba kroki mają $Res \neq \emptyset$. **TAK**.
- **possibly empty after** P — σ_0 spełnia *empty*. **TAK**.
- **necessary empty after** P — σ_1 spełnia $\neg empty$. **NIE**.
- **necessary $\neg hasA \wedge \neg hasB$ after** P — zarówno σ_0 , jak i σ_1 spełniają $\neg hasA \wedge \neg hasB$. **TAK** — niezależnie od przebiegu, po procesie żaden konsument nie ma produktu.